Elodias Adimou
développeur fullstack

DOCUMENTATION TECHNIQUE

Manuel technique

Takienta

Installation, conventions, exploitation et dépannage

Audience	Version	Date
Mainteneurs · Développeurs	v1.0 · Node 22 / pnpm 10 / Python 3.9+	Octobre 2026

Document établi par **Elodias Adimou** pour son projet personnel.

PRESTATAIRE

Elodias Adimou
Développeur web indépendant · Cotonou, Bénin
E-mail : nounagnonadimou@gmail.com
elodiasadimou.sabar.app · github.com/Elodias2001

Sommaire

1. Prérequis 3
2. Mise en route locale 4
3. Variables d'environnement 4
4. Scripts du gestionnaire de paquets 5
5. Structure du code 5
6. Conventions 6
7. Ajouter ou corriger une donnée 7
8. Autres briques 7
9. Opérations propres au projet 8
10. Tests 8
11. Build et déploiement 9
12. Observabilité opérationnelle 10
13. Sécurité opérationnelle 10
14. Procédures courantes 11
15. Dépannage 11
16. Mise à niveau 12
17. Ressources 12
18. Contacts internes 12
Annexe : points non déterminés à date 13

02 · Manuel technique

Comment on travaille sur Takienta au quotidien : installer, reconstruire les données, modifier le site, tester, publier, déployer, dépanner. Public : mainteneurs et développeurs qui reprennent le projet. Le document d'architecture (01) décrit la construction ; celui-ci décrit les gestes.

Base relevée	dépôt `Takienta`, branche `publication` (publiée comme `main`), commit `15748a8` (08/10/2026). Relevé le 26/10/2026.
Méthode	commandes recopiées depuis `package.json`, les workflows, les scripts et la documentation du dépôt ; comptes de tests rapportés par les outils.
Points ouverts	marqués **Non déterminé à date** et récapitulés en annexe
Historique	v1.0 : première version

1. Prérequis

Outil	Version	Rôle	Source
Node	`⩾ 20` ; CI en 22	build du site, tests, paquet JavaScript	`package.json`, `verify.yml`
pnpm	10.28.2	gestionnaire de paquets (`corepack enable` dans le `Dockerfile`)	`package.json`
Python	`⩾ 3.9` ; CI en 3.9 et 3.13	pipeline de données, paquet Python, tests	`pyproject.toml`, `verify.yml`
Outils Python de développement	`jsonschema ⩾ 4.20`, `pytest ⩾ 8`, `build ⩾ 1.2`, `mypy ⩾ 1.10`	tests, build, typage	`requirements-dev.txt`
Docker avec buildx	non épinglée	image du site	`Dockerfile`
`pdftotext` (poppler)	non épinglée	extraction de l'annexe de la loi, à la main seulement	`scripts/extraire-annexe-2013-05.py`
Chrome ou Chromium	non épinglée	rendu des PNG (vignettes, marque)	`scripts/chrome.mjs`
`sips`	livré avec macOS	fabrication de l'ICO de la marque	`scripts/rendre-marque.mjs`
`shasum`	livré avec macOS et Linux	un test compare l'empreinte de la loi archivée	`site/site.test.mjs`

2. Mise en route locale

Parcours le plus court, tel qu'il figure dans le README et dans `docs/01-demarrage.md` :

```
git clone https://github.com/Elodias2001/Takienta.git
cd Takienta
pnpm install
pnpm --filter benin-geo build
```

Pour construire et tester le site :

```
pnpm build:site
pnpm test:site
```

`pnpm build:site` écrit `dist-site/`. Pour voir le site dans une image locale : `docker compose up --build`, puis `docker compose port takienta-site 8080` donne le port publié (commentaire de `docker-compose.yml`).

Il n'y a ni compte administrateur, ni jeu de démonstration, ni formulaire de premier accès : le projet n'a ni base ni backend. La création de l'environnement Python local (`.venv/`) n'est écrite nulle part dans le dépôt **Non déterminé à date** ; la CI installe les outils avec `python -m pip install -r requirements-dev.txt`.

3. Variables d'environnement

Le dépôt ne contient ni `.env` ni `.env.example`. Six variables sont lues, aucune au moment du build du site.

Variable	Lue par	Rôle	Obligatoire
`TAKIENTA_CHROME`	`scripts/chrome.mjs`	chemin de Chrome pour les rendus PNG	non (candidats par défaut sur macOS et Linux)
`SITE_HOST`	`docker-compose.prod.yml`	nom d'hôte du site dans les étiquettes Traefik ; posée dans le `.env` du serveur, jamais dans le dépôt	oui en production
`COMPOSE_PROJECT_NAME`	compose de base et de production	préfixe du conteneur et des routeurs Traefik	non (défaut `takienta`)
`GHCR_OWNER`	`docker-compose.prod.yml`	propriétaire dans le nom de l'image	non (défaut `elodias2001`)
`IMAGE_TAG`	`docker-compose.prod.yml`	étiquette de l'image	non (défaut `main`)
`PYTHONPATH`	`site/site.test.mjs`	donne accès au paquet Python aux tests du site	posée par le test lui-même

Aucune variable n'est déclarée sans être lue, et aucune variable publique n'est incorporée au site. Les valeurs ne figurent pas dans ce document.

4. Scripts du gestionnaire de paquets

Racine (`package.json`) :

Script	Commande	Rôle
`data`	`node scripts/build-data.mjs`	écrit les modules de données des deux paquets
`build`	`pnpm -r build`	construit les paquets du dépôt
`test`	`pnpm -r test`	tests des paquets (le paquet JavaScript se reconstruit avant)
`type-check`	`pnpm -r type-check`	vérification des types
`build:brand`	`node scripts/build-brand.mjs`	SVG de marque, `palette.css` , `brand/README.md`
`build:carte`	`node scripts/build-carte.mjs`	cartes SVG des départements et des communes
`build:og`	`node scripts/build-og.mjs`	vignettes de partage (Chrome requis)
`build:site`	`node scripts/build-site.mjs`	site dans `dist-site/`
`test:site`	`vitest run`	tests du site

Paquet JavaScript (`packages/js/package.json`) : `data` (`node ../../scripts/build-data.mjs`), `build` (`node ../../scripts/build-data.mjs && tsup`), `type-check` (`build-data.mjs && tsc --noEmit`), `test` (`pnpm build && vitest run`).

> **Pas de Makefile, pas de migrations**
>
> Le dépôt n'a pas de `Makefile` : les commandes ci-dessus et les scripts de `scripts/` sont les seuls points d'entrée. Il n'a pas non plus de base de données, donc pas de migration.

5. Structure du code

```
Takienta/
  assets/carte/          cartes SVG, contours archivés, source*.json, README
  brand/                 tokens.json, SVG, PNG, polices, palette.css
  data/                  données canoniques, schémas, sources archivées, calque des pôles
  docs/                  5 fiches et un index publiés sur le site ; cadrage/ non publié
  packages/js/           paquet benin-geo (JavaScript)
  packages/python/       paquet benin-geo (Python)
  scripts/               pipeline, générateurs, rendus, contrôles
  site/                  générateur du site, pages, tests (site.test.mjs)
  tests/                 tests pytest de la racine
  .github/workflows/     ci.yml, verify.yml
  Dockerfile, nginx.conf, nginx-securite.conf, docker-compose.yml, docker-compose.prod.yml
```

Fichiers générés, à ne jamais éditer

Fichier ou dossier	Produit par
`packages/js/src/generated/`, `packages/python/src/benin_geo/_generated.py` (ignorés par Git)	`scripts/build-data.mjs`
`data/departements.json`, `communes.json`, `arrondissements.json`, `poles.json`	`scripts/construire-donnees.py`
`brand/svg/*`, `brand/palette.css`, `brand/README.md`	`scripts/build-brand.mjs` (`--verifier` échoue si un fichier diffère)
`assets/carte/benin-departements.svg`, `benin-communes.svg`, `README.md`	`scripts/build-carte.mjs` (`--verifier` disponible)
`dist-site/`, `packages/*/dist` (ignorés par Git)	`pnpm build:site`, `tsup`, `python -m build`
PNG et ICO de la marque, vignettes `site/img/og/*.png` (commités)	rendus à la main : `scripts/rendre-marque.mjs`, `pnpm build:og`

Corriger une donnée, c'est corriger `data/` (ou sa source) puis relancer le pipeline, jamais le fichier généré.

6. Conventions

Sujet	Règle relevée
Langue	Français partout (commentaires, documentation, commits, site) ; identifiants de code en anglais quand c'est plus lisible ; noms de domaine en français. Aucun tiret cadratin dans les textes publiés.
Commits	Convention `feat(data):`, `fix(core):`, `docs:`, `chore:`, rédigés en français (exemples du dépôt : `feat(site): …`, `chore(infra): …`). Rien dans le dépôt n'impose cette convention par un outil : pas de commitlint, pas de hook Git.
Identité Git	Dépôt personnel, identité locale `Elodias (Perso)`. Avant un commit : `git var GIT_AUTHOR_IDENT`. Ne pas pousser et ne pas déployer sans demande de l'autrice.
Branches	La branche de travail locale est `publication` ; elle est publiée comme `main` (`git push origin publication:main`). La CI se déclenche sur `main`.
TypeScript	`tsconfig.base.json` : `strict`, `noUncheckedIndexedAccess`, `noImplicitOverride`, `verbatimModuleSyntax`, cible ES2022. Python : `mypy --strict` sans erreur.
Outils absents	Pas d'ESLint, de Prettier, de `CODEOWNERS` ni de modèle de demande de fusion dans le dépôt.
Charte du site (testée)	Aucune couleur écrite en dur dans le CSS (variables `--tk-*`) ; aucun trait de 1 px ; ombres pleines, jamais diffuses ; Caprasimo pour les titres seulement ; une grille à une colonne s'écrit `minmax(0,1fr)`.

Sujet	Règle relevée
Données	Aucune donnée sans source déclarée dans `data/sources.json`. Un article de presse n'est pas une source. On n'ajuste jamais une valeur attendue pour faire passer un test. Tout nom entrant est contrôlé contre une liste de référence. Un identifiant n'est jamais modifié ni réutilisé.
Garde-fou de confidentialité	`python3 scripts/verifier-termes-prives.py` lit une liste de termes tenue hors dépôt ; l'option `--historique` contrôle l'historique et `--ref <branche>` limite le contrôle à la branche poussée. Sans la liste, le script s'arrête avec le code 2.

7. Ajouter ou corriger une donnée

Il n'y a pas de « collection » à déclarer : les données sont des fichiers produits par le pipeline. La procédure, d'après `docs/05-contribuer.md` et `AGENTS.md` :

1. S'appuyer sur un texte : loi, décret ou publication officielle.
2. Ajouter le document dans `data/sources/` avec son empreinte SHA-256, puis le déclarer dans `data/sources.json`.
3. Reconstruire : `python3 scripts/construire-donnees.py`, puis `node scripts/build-data.mjs`.
4. Lancer les tests (§10).

> **Prudence**
>
> Ne jamais régénérer ni éditer les deux instantanés d'identifiants (`data/identifiants.snapshot.json`, `data/identifiants-arrondissements.snapshot.json`). Un arrondissement créé plus tard s'ajoute en fin de liste de sa commune ; une suppression laisse un trou. Ne jamais ajouter un accent de mémoire à un libellé : un accent n'est retenu que si un village du même arrondissement l'atteste.

Un alias de commune ne s'ajoute que s'il est relevé dans une source. Les six libellés de la loi qui diffèrent du nom usuel vivent dans `data/rapprochement-communes.json`.

8. Autres briques

Ajout	Étapes
Une page du site	déposer un module dans `site/pages/` ; l'inscrire dans `ROUTES` de `scripts/build-site.mjs` ; ajouter sa vignette dans `site/og.mjs` et lancer `pnpm build:og` (le build du site échoue sans vignette) ; l'ajouter au tableau `pages` de `site/site.test.mjs` ; lancer `pnpm test:site`
Une fiche de documentation	déposer `docs/NN-nom.md` ; l'ordre vient du préfixe numérique ; sa vignette `og-docs-*.png` est requise (`pnpm build:og`). Les nombres cités sont comparés aux données par les tests, et les exemples de code sont exécutés contre les vrais paquets
Un changement de recherche	modifier `chercher` dans `packages/js/src/index.ts` et dans `packages/python/src/benin_geo/__init__.py` ; sinon `tests/test_croise.py` échoue

Ajout	Étapes
Une couleur ou l'emblème	couleurs dans `brand/tokens.json`, forme dans `scripts/emblem.mjs` ; puis `node scripts/build-brand.mjs` et `node scripts/rendre-marque.mjs` ; les contrastes sont calculés par `tests/test_marque.py`
Un contour de carte	archiver le fichier dans `assets/carte/` avec son empreinte et sa source, puis `node scripts/build-carte.mjs` ; ne rien télécharger sans l'accord de l'autrice

9. Opérations propres au projet

Opération	Commande	Quand, précautions
Rejouer le pipeline	`python3 scripts/construire-donnees.py`	après un changement de source ; `--verifier` contrôle sans écrire
Régénérer les modules des paquets	`node scripts/build-data.mjs`	après un changement dans `data/`
Régénérer les cartes	`node scripts/build-carte.mjs`	contours ou données changés ; `--verifier` disponible
Régénérer la marque	`node scripts/build-brand.mjs`	`tokens.json` ou `emblem.mjs` changés ; `--verifier` disponible
Rendre les vignettes	`pnpm build:og`	un titre de page a changé ; Chrome requis ; les PNG sont à commiter
Rendre les PNG de marque	`node scripts/rendre-marque.mjs`	emblème changé ; Chrome et `sips` requis (macOS)
Extraire l'annexe de la loi	`python3 scripts/extraire-annexe-2013-05.py <transcription.pdf> <loi-2013-05.pdf> <sortie.json>`	à la main, une fois ; jamais dans le build ; ne jamais lire les libellés ni les nombres sur le scan océrisé, qui confond des chiffres
Contrôler les termes privés	`python3 scripts/verifier-termes-prives.py --historique --ref <branche>`	avant tout push

10. Tests

Comptes rapportés par les outils le 26/10/2026, sur le commit `15748a8`, arbre de travail propre :

Suite	Commande	Résultat
Site (vitest)	`pnpm test:site`	1 fichier, 315 tests passés
Paquet JavaScript (vitest)	`pnpm test` dans `packages/js` (reconstruit d'abord)	3 fichiers, 42 tests passés (`vitest run` seul)

Suite	Commande	Résultat
Tests de la racine (pytest)	`python -m pytest tests`	87 passés
Paquet Python (pytest)	`python -m pytest packages/python`	34 passés
Types Python	`python -m mypy --strict packages/python/src/benin_geo`	aucune erreur dans 5 fichiers sources
Données à jour	`python3 scripts/construire-donnees.py --verifier`	code de sortie 0

Répartition des 87 tests de la racine : `test_marque` 18, `test_carte` 17, `test_arrondissements` 16, `test_donnees` 14, `test_annexe_2013` 8, `test_schemas` 7, `test_tiers_non_suivis` 2, `test_instantane` 2, `test_croise` 2, `test_termes_prives` 1.

Particularités :

- `tests/test_croise.py` lit `packages/js/dist` : construire le paquet JavaScript d'abord.
- Les tests du site appellent `shasum`, `python3` et `node scripts/build-carte.mjs --verifier`.
- `test_termes_prives` est ignoré tant que la liste locale de termes est absente.
- Aucun test ne pilote un navigateur, et la couverture de code n'est pas mesurée.

La CI enchaîne ces suites sur Python 3.9 et 3.13 (architecture, §4.4).

11. Build et déploiement

11.1 Build

Site : `pnpm build:site`. Image locale : `docker compose up --build`. Image de production, pour le serveur (architecture x86_64) depuis un Mac arm64 : `docker buildx build --platform linux/amd64 -t ghcr.io/elodias2001/takienta-site:main --load .`. Cette dernière commande vient de la pratique de l'autrice, pas d'un fichier du dépôt.

11.2 Déploiement

Le déploiement est manuel : le dépôt n'a aucun workflow de déploiement, volontairement. Les adresses et accès du serveur ne sont pas reproduits ici. Ordre à respecter :

1. Le DNS du domaine existe avant le démarrage du conteneur, sinon la validation Let's Encrypt de Traefik échoue.
2. L'image `linux/amd64` est transférée sur le serveur (`docker save` compressé, puis `docker load` à l'arrivée), sans registre.
3. Le fichier `docker-compose.prod.yml` du serveur est celui du dépôt ; il est comparé avant de le remplacer.
4. Dans le dossier du projet sur le serveur :
   `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`, avec `--force-recreate` pour recréer le conteneur à l'identique.

> **Prudence**
>
> Le serveur est partagé avec d'autres projets. Le déploiement ne touche que le dossier et le conteneur de Takienta, jamais Traefik ni un autre projet. Une chaîne de commandes qui enchaîne le build et le transfert avec `&&` peut recharger l'ancienne image si le build échoue au milieu d'un `tail` : vérifier le code de sortie du build avant le transfert.

11.3 Vérifications après déploiement

- `docker inspect takienta-site --format '{{.HostConfig.Memory}} {{.HostConfig.MemorySwap}}'` : 134217728 et 201326592 attendus (128 Mo et 192 Mo).
- L'état du conteneur est `running` et son contrôle de santé `healthy`.
- Les pages `/`, `/carte`, `/comparer` et `/provenance` répondent 200 ; `curl -sI` montre les en-têtes de sécurité.

11.4 Retour arrière

Aucun mécanisme n'est écrit : l'étiquette `main` désigne la dernière image chargée. La procédure de retour à une image précédente n'est pas définie Non déterminé à date.

11.5 Publier les paquets

La publication est manuelle et se fait avec un compte et un jeton personnels, conservés hors dépôt. Pratique relevée le 09/10/2026 :

```
# JavaScript, depuis packages/js
npm publish --access public

# Python, depuis packages/python
python -m build
twine upload dist/*
```

`twine` n'est pas dans `requirements-dev.txt`. Après publication, installer le paquet depuis le registre dans un dossier vide et appeler `chercher` pour vérifier. Quand un registre est publié, passer l'indicateur correspondant de `publie` à `true` dans `site/config.mjs` et écrire la ligne d'installation dans `docs/01-demarrage.md` : un test échoue sinon.

12. Observabilité opérationnelle

Les journaux sont ceux de Docker : `docker logs takienta-site`. Le contrôle de santé est `GET /health` (nginx répond `200 ok` sans journal d'accès). Il n'y a ni alerte, ni métrique, ni analytics. `site/config.mjs` annonce une rotation des journaux à 30 Mo cumulés (`json-file`, 10 Mo, 3 fichiers) ; le compose ne configure pas la journalisation, et le réglage qui s'applique n'est pas établi Non déterminé à date.

13. Sécurité opérationnelle

- Aucun secret dans le dépôt ; le dossier `local-secrets/` est ignoré par Git. Ne jamais copier la valeur d'un jeton dans un fichier suivi, une note ou un message.

- Les jetons npm et PyPI sont personnels et restent sur le poste de travail. Un jeton utilisé une fois dans un outil doit être révoqué ou limité au projet.
- En-têtes HTTP : voir l'architecture (§4.3). HSTS n'est pas envoyé et la version de nginx est visible.
- Aucun audit de dépendances n'est défini (ni commande, ni job).
- Le site n'envoie aucun script ; toute évolution qui introduirait une collecte, un cookie ou un prestataire entraîne la mise à jour de la page de confidentialité et de sa date.

14. Procédures courantes

Besoin	Comment
Vérifier que `data/` est à jour	`python3 scripts/construire-donnees.py --verifier`
Vérifier marque et cartes	`node scripts/build-brand.mjs --verifier` et `node scripts/build-carte.mjs --verifier`
Recalculer l'empreinte de la loi 2013-05	`shasum -a 256 data/sources/loi-2013-05.pdf`, à comparer à la valeur de la page Provenance et de `data/sources.json`
Tout reconstruire et tester	`python3 scripts/construire-donnees.py`, `node scripts/build-data.mjs`, `pnpm --filter benin-geo test`, puis `python -m pytest tests` et `python -m pytest packages/python`
Imposer un Chrome pour les rendus	`TAKIENTA_CHROME=/chemin/vers/chrome` devant la commande

15. Dépannage

Symptôme	Cause	Action
« Aucun Chromium trouvé »	aucun Chrome aux emplacements prévus	définir `TAKIENTA_CHROME`
Le build du site échoue avec « Vignette absente »	une page ou une fiche n'a pas son PNG de partage	`pnpm build:og`, puis commiter le PNG
`construire-donnees.py` s'arrête (« source absente du registre », « commune appariée deux fois »…)	donnée sans source, ou rapprochement incohérent	corriger `data/sources.json` ou `data/rapprochement-communes.json`
`build-carte.mjs` échoue (« commune non rattachée de façon unique »)	un nom de contour ne correspond à aucune commune	ajouter un rapprochement écrit dans `assets/carte/source-communes.json`
`test_croise` échoue	`chercher` modifié d'un seul côté, ou `dist` JavaScript absent	aligner les deux paquets ; construire le paquet JavaScript d'abord
`verifier-termes-prives.py` sort avec le code 2	la liste locale de termes est absente	recréer `local-secrets/termes-prives.txt`

Symptôme	Cause	Action
Une grille déborde à 375 px	`1fr` au lieu de `minmax(0,1fr)`	corriger la déclaration de la grille
Pas de certificat TLS au premier démarrage	le DNS n'existait pas au démarrage du conteneur	créer le DNS, puis recréer le conteneur

16. Mise à niveau

Les versions sont épinglées par `pnpm-lock.yaml` (`--frozen-lockfile` en CI et dans le `Dockerfile`) ; TypeScript reste en 5.x. Les deux versions de paquet (`packages/js/package.json` , `packages/python/pyproject.toml`) et `__version__` dans `__init__.py` doivent rester identiques : `tests/test_croise.py` les compare. La pratique de mise à jour des dépendances (au cas par cas ou selon une procédure) n'est pas écrite **Non déterminé à date** .

17. Ressources

Document	Contenu
`README.md`	présentation, niveaux de données, démarrage
`docs/01..05-*.md`	démarrage, référence de l'API, modèle de données, identifiants, contribuer (publiées sur le site)
`docs/cadrage/PROMPT-TAKIENTA.md`	cadrage du projet
`docs/cadrage/recherche-nom-takienta.md`	recherche sourcée sur le nom
`AGENTS.md`	garde-fous et décisions figées
`brand/README.md` , `assets/carte/README.md`	marque et cartes (générés)
`data/sources/NOTES-SOURCING-2026-09-28.md`	notes sur l'état des sources officielles

18. Contacts internes

Le dépôt ne contient pas de `CODEOWNERS` . Mainteneuse et seule personne identifiée : Elodias Adimou. Aucun canal d'astreinte n'est écrit **Non déterminé à date** .

Annexe : points non déterminés à date

Ces points ne se déduisent ni du code ni des constats disponibles. Ils seront complétés dans une prochaine version du document.

#	Point	Section
1	Commande de création de l'environnement Python local (`.venv/`)	§2
2	Retour arrière vers une image précédente	§11.4
3	Réglage réel de la rotation des journaux Docker sur le serveur	§12
4	Pratique de mise à jour des dépendances	§16
5	Canal d'astreinte et contacts autres que la mainteneuse	§18